

Hash Functions

Input m : binary string

Output $h(m)$:

- "Short" (n-bit) binary string (aka **message digest** or hash)

Efficiently computable

Applications: cryptography, security, efficiency

Keyed $h_k(m)$, where the key is public, or unkeyed $h(m)$

Motivation: Hashing for Efficiency

Input: large set (e.g. ints or strs)

Goal: map "randomly" to few bins

Security Goal: Collision Resistance:

A **collision**: two inputs (e.g. names) with the same hash: $h('Bob') = h('Phil')$

Every hash has collisions, since $|\text{input}| \gg |\text{output}|$

Collision resistance: hard to **find** these collisions

- Note: attacker can always try names randomly until a collision is found
- But this should be ineffective: must try about (on average) N names (number of bins)

Collision Resistant Hash Function (CRHF)

h is CRHF if it is hard to find collisions $h(x) = h(x')$ for $x \neq x'$

- Note: attacker can always try names randomly until collision is found
- But this should be ineffective: must try about $|Range|$ values

Definition (Keyless Collision Resistant Hash Function (CRHF)). A keyless hash function $h^n(\cdot) : \{0, 1\}^* \rightarrow \{0, 1\}^n$ is *collision-resistant* if for every efficient (PPT) algorithm A , the advantage $\varepsilon_{h,A}^{CRHF}(n)$ is negligible in n , i.e. smaller than any positive polynomial for sufficiently large n (as $n \rightarrow \infty$), where:

$$\varepsilon_{h,A}^{CRHF}(n) = \Pr[(x, x') \leftarrow A(1^n) \text{ s.t. } (x \neq x') \wedge (h^n(x) = h^n(x'))]$$

Keyless CRHF Do Not Exist

$|Range| \ll |Domain|$ so there is a collision where $h(x') = h(x), x \neq x'$

Keys are powerful because they allow you to change the terms on your adversary.

For a keyless CRHF there is a PPT algorithm A that can always output a collision: $A(1^n) = \{return\ x, x'\}$

- Proof: intuitively, since the function is fixed (same input-output mapping), a collision instance can be hardcoded into the attacker algorithm and just output that collision and win the security game.

Keyed CRHF

$k \xleftarrow{\$} \{0, 1\}^n \leftarrow \text{Adversary} \leftarrow (x, x') \text{ If } x \neq x' \text{ and } h(x) = h(x')$

Generic Collision Attacks

An attacker that runs in exponential time can always find a collision (i.e. non-PPT attacker)

- Easy: find collisions in 2^n time by trying $2^n + 1$ distinct inputs (compute their hash and locate a collision)

An attacker finds a collision with 2^{-n} probability (negligible probability).

- Choose x and x' at random and check if they produce a collision.

The Birthday Paradox

The birthday paradox states that the expected number q of hashes (i.e. hash queries) until a collision is found in $O(2^{\frac{n}{2}})$ not $O(2^n)$. It is $q \lesssim 2^{n/2} * \sqrt{\frac{\pi}{2}} \lesssim 1.254 * 2^{n/2}$
For 80 bits of effective security, use $n = 160$

- So to defend against an attacker who can perform 2^{80} hashes set the digest length to at least 160 bits (so the range has a size of 2^{160} digests)

The Birthday Attack

Probability of NO birthday-collision:

- Two people: $(364/365)$
- Three people $(364/365)(364/365)$
- ...
- n people: $\prod_{i=1}^{n-1} \frac{365-i}{365}$
- So for 23 individuals, the probability of a birthday collision is about .5

Collision-Resistance: Applications

Integrity (of object/file/message)

- Send $hash(m)$ securely to validate m
- Later, we will see how a hash function can be used to construct a MAC (called HMAC)

Hash-then-Sign

- Instead of signing m , sign $hash(m)$ (more efficient!)

Blockchains

Other Notions of Security

Collision resistance provides the strongest guarantee

- Gives more freedom to the adversary; the adversary always wins if it finds any two inputs with the same digest.
- No conditions on these two inputs other than being in the domain of the hash function.

Hash Function: is a mapping that takes an input (VIL) and maps it to a fixed length n string where n is the security parameter.

Other security notions (but sufficient for many applications):

- Target collision resistance (TCR)
- Second preimage resistance
- First preimage resistance

Birthday paradox (or attack) does not work against these weaker notions. It is for collision resistance; find *any* two inputs that collide!

TCR and Birthday Paradox?

TCR: Adversary has to select the target *before* knowing the key

We (mostly) focus on keyless hash

Although there are no CRHFs and theory papers focus on keyed hash

But...

- It's a bit less complicated and easier to work with.
- No need to consider both ACR (Any Collision Resistance, which is the same as CRHF) and TCR. Why?
- Modifying to CRHF is quite trivial: just make it keyed
- Usually used in practice: code libraries, standards, ...

2nd-Preimage-Resistant Hash (SPR)

$y = h(x)$; image = preimage

Hard to find collision with a specific random x

So, the attacker's advantage (probability of finding such a collision) in the following game is negligible.

Here, adversary has no freedom over the first preimage x , whereas in TCR, the attacker can choose x (and then the second x is after knowing the hash function)

Example: hashing software from developer is SPR, since the software hash is like the first preimage. The attacker must find a collision against this particular message (e.g. the software)

CRHF/SPR vs. Applications

CRHF secure for signing, SW-distribution

How about SPR hash (weak CRHF)?

- SW-distribution? YES
- Hash-then-sign? NO

Why?

Attacker can't impact SW to be distributed, but the attacker may be able to impact the signed message!

Since h is not necessarily CRHF, its signature can be forged (the tag for two collisions is the same)

SPR: Collisions to Chosen Messages

Or: Alice and Mal, the corrupt lawyer

Mal finds two "colliding wills," GoodW and BadW:

- GoodW: contents agreeable to Alice
- $h(\text{GoodW}) = h(\text{BadW})$
- Alice signs good will: $\text{Sign}(h(\text{GoodW}))$
- $\text{Sign}(h(\text{GoodW})) = \text{Sign}(h(\text{BadW}))$

Is such an attack realistic? Or is SPR enough "in practice?" Yes.

We usually rely on having a specific format for messages, like a fixed prefix chosen by the attacker.

Examples

1. Let h_k be a keyed CRHF. Is $h'_k = h_k(h_k(x))$ a CRHF? Why?
2. For x parsed as $x = x_1 || x_2 || x_3$, let $h(x_1 || x_2 || x_3) = x_1 + x_2 + x_3 \% p$, is h a CRHF? Why? Is it an SPR? Why?
3. Let $h_k(m)$ be a TCR function. Construct $h'_k(m) = 0^n$ if $m[1 : |k|] = k$ and $h_k(m)$ otherwise (recall that the hash function input can be of any length). Is h'_k a CRHF? Why? Is h'_k a TCR? Why?
 - Not a CRHF: A knows k before outputting m and m' , A can pick any two messages where the first $|k|$ bits are the key.
 - TCR: Attacker doesn't know key beforehand. m chosen in advance must start with the key for the attacker to be successful, but the chances of this happening are $\frac{1}{2^{|k|}}$, which is negligible.

One-Way Function (OWF/First Preimage Resistance)

One-way function or first preimage resistance: given $h(k)$ for a random x , it is hard to find x , or any x' s.t. $h(x') = h(x)$ (like inverting the hash func)

Given the output, cannot find the input (or a collision).

Compare to:

- Collision-Resistance (CRHF): hard to find any collision, i.e. any (x, x') s.t. $h(x') = h(x), x \neq x'$
- Second-Preimage Resistance (SPR): hard to find a collision with random x , i.e. x' s.t. $h(x') = h(x), x \neq x'$

Application: One-time Password Authentication

Used with OWF, since it can't be inverted to glean passwd

One-time password auth:

- Select random x : "one-time password" (keep secret)
- Validate using non-secret "one-time validation token:" $h(x)$

Randomness Extraction

It is a desired property of hash funcs.

"If input is sufficiently random, then the output is random"

Randomness extraction: if any m input bits are random $\rightarrow$ all n output bits are pseudorandom, for sufficiently large m

ROM (Random Oracle Model): assume hash function behaves as a true random function (difference from randomness extraction, which is not an assumption)

Exercise

Let h_1, h_2 be both CRHF and OWF

Use them to construct h_{CRHF} : CRHF but not OWF and h_{OWF} but not CRHF

One possible solution:

•

$$h_{CRHF}(m) = \begin{cases} m & \text{if } |m| = n \\ h_1(m) & \text{otherwise} \end{cases}$$

Since adversary selects the length of the message, it can set it to length n which forces it to branch to 1 every time in the security game.

•

$$h_{OWF}(m) = \begin{cases} h_1(m) & \text{if } |m| = n \\ h_1(m_{1..n} \oplus h_2(m')) & \text{if } m = m_{1..n} || m' \end{cases}$$

Not OWF. Once an attacker computes an arbitrary $h_2(m')$, they can find an $m_{1..n}$ that produces the same output when $\oplus$ 'd with $h_2(m')$.

Security Notions

CRHF > TCR > SPR

OWF is separate to all of these.